

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192412015
Name	K.S.Dharshana

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I K.S.Dharshana(192412015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Question 1: AI-Powered Drone for Emergency Medicine Delivery Scenario

A government deploys AI-powered drones to deliver emergency medicines in flood-affected regions. The environment is highly dynamic because roads may suddenly become inaccessible, weather conditions change frequently, and movement costs vary depending on terrain. The drone has access to heuristic estimates (straight-line distance), partial real-time updates about blocked paths, and variable travel costs. Therefore, it must quickly determine a safe and efficient route with minimum delay.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next node based only on the heuristic value $h(n)$, which estimates the distance from the current location to the destination. It always expands the node that appears closest to the goal without considering the actual travel cost.

In this scenario, GBFS allows the drone to make decisions very quickly, making it suitable for emergency medicine delivery where response time is critical. However, since it ignores factors such as weather conditions, flooded roads, and terrain cost, it may choose routes that appear shorter but are actually unsafe or expensive. Therefore, GBFS is fast but cannot always guarantee the best or safest path in uncertain environments.

ii. A Search Algorithm*

A* Search combines both the actual travel cost and the heuristic estimate to determine the most efficient path. Its evaluation function is:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual cost from the start node to the current node.
- $h(n)$ = Estimated cost from the current node to the goal.

Unlike Greedy Search, A* considers both the distance already travelled and the estimated remaining distance. This enables the drone to avoid high-cost or risky routes while still moving efficiently toward the destination. If new information such as blocked roads or severe weather becomes available, A* can update the path and continue searching for a better alternative. Although A* requires more computation, it produces safer and more optimal routes.

iii. Impact of Heuristic Accuracy

The performance of both algorithms depends on the quality of the heuristic.

When the heuristic is highly accurate, Greedy Best-First Search reaches the destination quickly because it selects the node closest to the goal. A* also becomes faster by exploring fewer unnecessary nodes while still maintaining optimality.

However, when the heuristic is inaccurate, Greedy Search may repeatedly choose poor routes, resulting in longer travel times or unsafe paths. In contrast, A* is more reliable because it also considers the actual travel cost, allowing it to recover from heuristic errors and still find a near-optimal solution.

iv. Effect of Dynamic Changes

Flood situations change continuously due to blocked roads, damaged bridges, heavy rainfall, and strong winds. These dynamic changes directly affect route planning.

Greedy Best-First Search performs poorly in such situations because it relies only on heuristic information and may continue toward blocked paths, requiring frequent restarts. A* handles dynamic environments more effectively by recalculating path costs whenever new information becomes available. As a result, it can adapt to environmental changes while maintaining efficient navigation.

v. Recommended Algorithm

Among the two algorithms, *A Search** is the most suitable for emergency drone delivery. Although Greedy Best-First Search provides faster decisions, it ignores actual travel costs and cannot guarantee safe navigation. A* balances speed, optimality, and safety by considering both the real travel cost and heuristic estimate. This enables the drone to avoid dangerous routes, respond to changing environmental conditions, and deliver medicines with minimum delay and maximum reliability.

Conclusion

For emergency medicine delivery in flood-affected regions, *A Search** is the preferred algorithm because it provides optimal route planning, adapts to dynamic environmental changes, and ensures safer navigation. While Greedy Best-First Search offers faster decisions, A* achieves a better balance between real-time performance, optimality, and operational safety, making it the ideal choice for this application.

Question 2: Smart City Traffic Signal Optimization

Scenario

A smart city implements an AI-based traffic management system to optimize signal timings across hundreds of intersections. The main objectives are to minimize **total waiting time, fuel consumption, and traffic congestion**. Since traffic patterns continuously change throughout the day, the system must adjust signal timings dynamically. However, the search space is extremely large, and the system may become trapped in locally optimal solutions. Therefore, optimization techniques are required instead of traditional search methods.

Problem Formulation

The traffic signal optimization problem can be formulated as an **optimization problem** where the objective is to find the best signal timing configuration that minimizes waiting time, fuel consumption, and congestion while maximizing traffic flow.

Each possible signal timing represents a **state**, and changing the green or red signal duration represents moving from one state to another. Since there are hundreds of intersections and numerous timing combinations, the number of possible solutions becomes extremely large. Traditional search algorithms such as Breadth-First Search (BFS) or Depth-First Search (DFS) are inefficient because they attempt to explore a large number of possible states. This requires excessive computation time and memory, making them unsuitable for real-time traffic management. Therefore, optimization algorithms are preferred because they search for good solutions without exploring every possibility.

i. Hill Climbing Algorithm

Hill Climbing is a local search algorithm that continuously improves the current solution by selecting the neighboring solution with the highest improvement.

Working Process

1. Start with an initial traffic signal timing.
2. Evaluate the performance using waiting time and congestion.
3. Generate neighboring signal timings by making small adjustments.
4. Select the neighboring solution that improves traffic flow the most.
5. Repeat the process until no better neighboring solution exists.

Advantages

- Simple and easy to implement.
- Requires very little memory.
- Produces quick solutions.
- Suitable for small optimization problems.

Limitations

- Cannot guarantee the global optimum.
- May stop at a local optimum.
- Cannot easily escape poor solutions.
- Performance decreases for complex traffic networks.

ii. Local Maxima, Plateaus, and Ridges

Hill Climbing often encounters several real-world problems during optimization.

Local Maximum

A local maximum is a solution that is better than its neighboring solutions but is not the best overall solution.

Real-world Example:

Improving signal timing at one busy intersection may reduce congestion there but increase traffic jams at nearby intersections.

Plateau

A plateau occurs when several neighboring solutions have nearly identical performance, making it difficult for the algorithm to determine the best direction.

Real-world Example:

Increasing or decreasing the green signal by one second produces almost no noticeable improvement in traffic flow.

Ridge

A ridge occurs when improvement requires multiple changes simultaneously instead of a single change.

Real-world Example:

Changing the signal timing at one intersection alone has little effect, but adjusting several neighboring intersections together significantly improves traffic movement.

iii. Simulated Annealing and Genetic Algorithms**Simulated Annealing**

Simulated Annealing is an optimization algorithm that occasionally accepts worse solutions during the search. This allows it to escape local maxima and continue searching for better global solutions. As the search progresses, the probability of accepting worse solutions gradually decreases.

Advantages

- Escapes local optima.
- Produces better-quality solutions.
- Suitable for dynamic optimization problems.

Genetic Algorithm

Genetic Algorithm is inspired by the process of natural evolution.

Working Steps

1. Generate an initial population of signal timing solutions.
2. Evaluate each solution using a fitness function.
3. Select the best-performing solutions.
4. Perform crossover to combine good solutions.
5. Apply mutation to introduce variation.
6. Repeat until an optimal or near-optimal solution is obtained.

Advantages

- Explores multiple solutions simultaneously.
- Avoids local optimum problems.
- Highly scalable.
- Produces high-quality solutions for large search spaces.

iv. Recommended Approach

Among the optimization techniques, the **Genetic Algorithm (GA)** is the most suitable for smart city traffic management.

Justification

- Handles extremely large search spaces efficiently.
- Optimizes multiple objectives simultaneously.
- Adapts to continuously changing traffic conditions.
- Produces near-optimal solutions within a reasonable time.
- Scales effectively for hundreds of intersections.

Although Hill Climbing is simple and fast, it frequently gets trapped in local optimum solutions. Simulated Annealing performs better by escaping local optima, but Genetic Algorithms provide greater scalability and adaptability for large, real-time traffic systems.

Conclusion

Traffic signal optimization is a complex optimization problem that cannot be solved efficiently using traditional search techniques. While Hill Climbing provides quick improvements, it suffers from local maxima, plateaus, and ridges. Simulated Annealing overcomes some of these limitations, but **Genetic Algorithms** offer the best balance of scalability, adaptability, and optimization quality. Therefore, **Genetic Algorithm** is the most appropriate approach for developing an intelligent and efficient smart city traffic management system.

Question 3: Autonomous Mars Rover – Online Search Agent

Scenario

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. The rover must navigate safely while avoiding hazards such as rocks and craters. Since it has limited sensing capability and does not possess a complete map of the environment, it must make decisions in real time by continuously updating its knowledge as it explores. This makes the problem suitable for an **Online Search Agent**.

i. Why the Problem Requires an Online Search Agent

Unlike offline search, where the complete environment is known before the search begins, the Mars rover operates in an unknown and partially explored environment. It cannot predict all obstacles or terrain conditions in advance.

An **Online Search Agent** makes decisions while interacting with the environment. The rover observes its surroundings, chooses the next action, and updates its knowledge after every movement. This enables it to continue exploring safely even without a complete map.

Therefore, online search is more suitable because it supports **real-time decision-making**, **continuous learning**, and **adaptive navigation** in unknown environments.

ii. Challenges of Partial Observability and Dynamic Environments

Partial Observability

The rover has limited sensors and can observe only a small portion of the environment at a time. It cannot detect distant obstacles or hidden hazards until it moves closer.

Examples:

- Hidden craters
- Large rocks
- Sand pits
- Uneven terrain

As a result, the rover must make decisions with incomplete information.

Dynamic Environment

Although Mars changes slowly, environmental conditions such as dust storms, shifting sand dunes, and uneven surfaces can affect navigation.

These changes may:

- Block previously safe paths.
- Increase travel time.
- Affect wheel movement and energy consumption.

The rover must continuously adapt its route to ensure safe exploration.

iii. Building and Updating the Internal Model

The rover maintains an **internal model** of its surroundings based on the information collected through its sensors.

Working Process

1. Sense the nearby environment using onboard sensors.
2. Identify obstacles, rocks, slopes, and safe paths.
3. Update the internal map with newly discovered information.
4. Store previously visited locations.
5. Recalculate the safest route whenever new obstacles are detected.

By continuously updating this internal model, the rover gradually builds a more accurate representation of the Martian surface, allowing it to make better navigation decisions.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Uninformed Search	Informed Search	Online Search
Environment Knowledge	Complete	Complete with heuristic	Partial
Uses Heuristic	No	Yes	May use available information
Adaptability	Low	Moderate	High
Real-Time Decision Making	No	Limited	Yes
Suitable for Mars Rover	No	Partially	Yes

Uninformed search algorithms such as BFS and DFS require complete knowledge of the environment and are unsuitable for unknown terrains. Informed search algorithms like A* improve efficiency using heuristic information but still assume that the environment is mostly known. Online search is specifically designed for unknown and changing environments, making it the best choice for autonomous exploration.

v. Recommended Approach

The most suitable approach for the Mars rover is an **Online Search Agent** because it can operate effectively under uncertainty and limited information.

Justification

- Makes decisions in real time.
- Learns continuously from sensor data.
- Updates its internal map while exploring.
- Adapts to newly discovered obstacles and hazards.
- Ensures safe and efficient navigation.
- Suitable for unknown and partially observable environments.

Compared to offline, uninformed, and informed search strategies, online search provides greater flexibility and adaptability, making it ideal for autonomous planetary exploration.

Conclusion

The Mars rover navigation problem requires an **Online Search Agent** because the environment is unknown, partially observable, and continuously changing. By sensing the environment, updating its internal model, and making real-time decisions, the rover can safely navigate, avoid hazards, and successfully complete its exploration mission. Therefore, **Online Search** is the most appropriate and reliable approach for autonomous rover operations on Mars.

Question 4: University Exam Timetable Generation using Constraint Satisfaction Problem (CSP)

Scenario

A university plans to develop an AI-based system to automatically generate exam timetables. The system must satisfy several constraints such as preventing overlapping exams for students, managing limited exam halls, ensuring faculty availability, maintaining subject scheduling order, and providing special accommodations for certain students. As the number of students and courses increases, the complexity of generating a conflict-free timetable also increases. Hence, the problem can be effectively modeled as a **Constraint Satisfaction Problem (CSP)**.

i. Formulating the Problem as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem consists of **variables, domains, and constraints**.

Variables

Each variable represents an exam that needs to be scheduled.

Example:

- E1 – Mathematics
- E2 – Physics
- E3 – Artificial Intelligence
- E4 – Database Management Systems

Domains

The domain of each variable represents all possible values that can be assigned.

Example:

- Time Slots: Morning, Afternoon, Evening
- Exam Halls: Hall A, Hall B, Hall C

Each exam can be assigned to any available combination of time slot and hall.

Constraints

The timetable must satisfy the following constraints:

- A student should not have two exams at the same time.
- The same exam hall cannot be assigned to two exams simultaneously.
- Faculty members should be available during the scheduled examination.
- Certain subjects must be conducted before others (prerequisite constraints).
- Students requiring special accommodations must be assigned suitable examination halls.
- Hall capacity should not exceed the number of registered students.

Only when all these constraints are satisfied is the timetable considered valid.

ii. Backtracking Search

Backtracking Search is the most common algorithm used to solve CSPs. It systematically assigns values to variables while checking whether all constraints are satisfied.

Working Process

1. Select an unscheduled exam.
2. Assign an available time slot and exam hall.
3. Check whether all constraints are satisfied.
4. If no conflict exists, proceed to schedule the next exam.
5. If a conflict occurs, undo the current assignment (backtrack).
6. Try another available assignment.
7. Continue until every exam has been successfully scheduled.

Advantages

- Guarantees a valid solution if one exists.
- Easy to implement.
- Suitable for moderate-sized timetable problems.

Limitation

For very large universities with hundreds of courses and thousands of students, plain backtracking becomes slow because it explores many unnecessary possibilities.

iii. Constraint Propagation (Forward Checking)

Constraint propagation improves the efficiency of backtracking by reducing the search space.

One common technique is **Forward Checking**.

Whenever a variable is assigned, forward checking immediately removes incompatible values from the domains of the remaining variables.

Example

Suppose **Mathematics** is scheduled on **Monday Morning in Hall A**.

Forward Checking immediately removes:

- Monday Morning from all subjects having common students.
- Hall A from the list of available halls for the same time slot.

As a result, many invalid assignments are eliminated before they are explored.

Advantages

- Detects conflicts early.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.
- Improves overall efficiency.

iv. Real-World Challenges and Best Strategy

In practical situations, timetable generation becomes increasingly difficult due to several factors:

- Thousands of students enrolled in multiple courses.
- Limited examination halls.
- Faculty leave or sudden unavailability.
- Last-minute timetable modifications.
- Hall capacity restrictions.
- Special accommodations for differently-abled students.

To efficiently handle these challenges, a combination of **Backtracking Search** and **Constraint Propagation (Forward Checking)** is the best strategy.

Backtracking ensures that every constraint is satisfied, while Forward Checking reduces the search space by eliminating invalid choices at an early stage. This combination improves scalability, minimizes computation time, and generates conflict-free exam schedules even for large universities.

Conclusion

The university exam timetable problem is a classic **Constraint Satisfaction Problem (CSP)**. Modeling the problem using variables, domains, and constraints enables systematic scheduling. Combining **Backtracking Search** with **Forward Checking** provides an efficient and scalable solution capable of handling large numbers of courses, students, and real-world scheduling constraints.

Question 5: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Scenario

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, evaluate numerous possible moves, and make intelligent decisions within strict time limits. Since the number of possible game states grows rapidly, the decision tree becomes extremely large, making efficient search algorithms essential.

i. Minimax Algorithm

The **Minimax Algorithm** is a decision-making algorithm used in two-player games where one player attempts to maximize the score while the opponent attempts to minimize it.

In this scenario:

- The AI acts as the **Maximizing Player**.
- The Human acts as the **Minimizing Player**.

The algorithm explores all possible future moves and predicts the opponent's best response before selecting its own move.

Working Process

1. Generate all possible moves.
2. Expand the game tree.
3. Evaluate the final game states.
4. Propagate scores upward.
5. Choose the move with the highest guaranteed score.

This enables the AI to make rational decisions assuming the opponent also plays optimally.

ii. Computational Challenges

The main challenge of Minimax is the enormous size of the game tree.

For every possible move, multiple future moves are generated, causing exponential growth.

Challenges

- Very high branching factor.
- Large memory consumption.
- Long computation time.
- Difficult to satisfy real-time decision requirements.
- Millions of game states may need evaluation.

As game complexity increases, plain Minimax becomes computationally expensive.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique that significantly improves the efficiency of Minimax.

It eliminates branches that cannot influence the final decision.

Key Concepts

- **Alpha (α):** Best score currently guaranteed for the maximizing player.
- **Beta (β):** Best score currently guaranteed for the minimizing player.

Whenever:

$$\alpha \geq \beta$$

the remaining branches are ignored because they cannot produce a better outcome.

Advantages

- Reduces the number of nodes explored.
- Produces the same optimal result as Minimax.
- Faster execution.
- Lower memory usage.
- Suitable for real-time games.

Thus, Alpha-Beta Pruning allows the AI to search deeper within the same time limit.

iv. Evaluation Function

When the search cannot continue until the end of the game, the AI estimates the quality of a game state using an evaluation function.

Example Evaluation Function

$$\text{Score} = (5 \times \text{Resources}) + (10 \times \text{Army Strength}) + (8 \times \text{Territory Control}) - (6 \times \text{Enemy Strength})$$

Factors Considered

- Number of soldiers.
- Available resources.
- Territory controlled.
- Player health.

- Enemy strength.
- Strategic position.
- Defensive capability.

A higher evaluation score indicates a stronger position for the AI.

v. Recommended Strategy

The most suitable approach is **Minimax combined with Alpha-Beta Pruning**.

Justification

- Produces optimal decisions.
- Significantly reduces computation time.
- Explores fewer unnecessary game states.
- Meets strict real-time constraints.
- Improves scalability for complex strategy games.
- Widely used in modern Chess, Checkers, and Strategy Game AI.

By reducing the search space while maintaining optimality, Alpha-Beta Pruning enables intelligent decision-making without sacrificing performance.

Conclusion

For real-time strategy games, **Minimax with Alpha-Beta Pruning** provides the ideal balance between intelligence, efficiency, and speed. It allows the AI to evaluate complex game states quickly, make optimal decisions, and respond effectively within real-time constraints, making it the preferred technique for modern game AI systems.